Maximum k to Sort a Permutation Using Bitwise AND Swaps

Problem Statement

You are given an integer array nums of length n, where nums is a permutation of the numbers in the range [0..n - 1].

You may swap elements at indices i and j only if nums[i] AND nums[j] == k, where AND denotes the bitwise AND operation and k is a non-negative integer.

Return the maximum value of k such that the array can be sorted in non-decreasing order using any number of such swaps. If nums is already sorted, return 0.

Example 1:
Input: nums = [0,3,2,1]
Output: 1
Explanation: Choose k = 1. Swapping nums[1] = 3 and nums[3] = 1 is allowed since 3 AND 1 == 1, resulting in [0,1,2,3].

Example 2:
Input: nums = [0,1,3,2]
Output: 2
Explanation: Choose k = 2. Swapping nums[2] = 3 and nums[3] = 2 is allowed since 3 AND 2 == 2, resulting in [0,1,2,3].

Example 3:
Input: nums = [3,2,1,0]
Output: 0
Explanation: Only k = 0 allows sorting since no greater k allows the required swaps.

Constraints:
1 <= n == nums.length <= 10^5
0 <= nums[i] <= n - 1
nums is a permutation of integers from 0 to n - 1.

Approach

Key observation:

For a fixed k, consider a graph where each number from 0 to n - 1 is a node. There is an edge between a and b exactly when a AND b == k.

To sort the array, for every index i with nums[i] != i, the value i must be connected to the value nums[i] in this graph.

If k is not a subset of i, then i has no allowed swap partner:
i AND x can never equal k because i is missing at least one bit of k.

Therefore, for every non-fixed index i, k must be a subset of both i and nums[i]. Equivalently:

k must be a subset of i AND nums[i]

for every i where nums[i] != i.

The maximum possible k is therefore the bitwise AND of i AND nums[i] over all non-fixed indices.

Why this k always works:

Let k be that AND value. For every non-fixed index i, k is a subset of i and also a subset of nums[i].

Then:
- i AND k == k, so i can be swapped with k.
- nums[i] AND k == k, so nums[i] can be swapped with k.

Thus, i and nums[i] are connected through the value k. Since this holds for every misplaced value, the entire permutation can be sorted using k.

If the array is already sorted, there are no non-fixed indices, so return 0.

Examples:
- [0,3,2,1]: non-fixed pairs give 1 AND 3 = 1 and 3 AND 1 = 1, so answer = 1.
- [0,1,3,2]: non-fixed pairs give 2 AND 3 = 2 and 3 AND 2 = 2, so answer = 2.
- [3,2,1,0]: every non-fixed pair has bitwise AND 0, so answer = 0.

Java Solution

```
class Solution {
    public int sortPermutation(int[] nums) {
        int maximumK = -1;

        for (int index = 0; index < nums.length; index++) {
            if (nums[index] == index) {
                continue;
            }

            if (maximumK == -1) {
                maximumK = index & nums[index];
            } else {
                maximumK &= index & nums[index];
            }
        }

        return maximumK == -1 ? 0 : maximumK;
    }
}
```

Complexity

Time Complexity: O(n), since the array is scanned once.

Space Complexity: O(1), since only constant extra space is used.